



Insight Hive

TOTVS Meeting Intelligence — backend Java responsável por transformar transcrições de reuniões em achados acionáveis, estratégias de resolução e métricas de sucesso do cliente.

EQUIPE INSIGHT HIVE

Mateus Amaral	RM 562334	Enzo Hort Ramos	RM 561872
Eduardo Tolentino	RM 562169	João Pedro Palermo	RM 562077
Azor Tartuce	RM 563995		

Sumário

1	Objetivo e Escopo do Projeto	O problema, a proposta de solução e os limites do backend
2	Funcionalidades da Solução	Principais capacidades expostas pela API
3	Protótipo — Telas do Sistema	Mockups ilustrativos de como um cliente consome a API
4	Modelo do Banco de Dados	Diagrama entidade-relacionamento (Oracle) e tabelas
5	Diagrama de Classes Atualizado	Modelo de domínio (pacote <i>model</i>) e seus relacionamentos

Este documento foi produzido a partir da análise do código-fonte do repositório **sprintjava** (commit 3a46005) — README, docs/ARQUITETURA.md, schema.sql e classes de domínio, service, controller e DAO.

1 Objetivo e Escopo do Projeto

Contexto

O **Insight Hive** é a solução desenvolvida pela equipe para o Challenge 2026 (TOTVS) da disciplina Domain Driven Design – Java, sob o codinome interno *TOTVS Meeting Intelligence*. O produto ataca um problema recorrente em times de vendas e Customer Success: reuniões com clientes geram sinais valiosos (reclamações, riscos de churn, lacunas de adoção) que raramente são sistematizados — ficam presos em anotações soltas e não viram ação nem métrica.

Objetivo

Este repositório entrega o **backend Java (Spring Boot)** da solução: o serviço do meio entre o front-end (Next.js) e um motor de análise por IA (Python/Ollama). Ele recebe a transcrição de uma reunião, aciona a análise de IA, e transforma o resultado em **achados (findings)** rastreáveis — cada um passível de receber **tentativas de resolução** baseadas em um catálogo de **estratégias**, até ser fechado por uma **sugestão** confirmada pelo usuário. Ao final, um **dashboard** consolida a eficácia de cada estratégia e a saúde da carteira de clientes.

DENTRO DO ESCOPO DESTES REPOSITÓRIO

- API REST em Java 21 / Spring Boot 3.3.4, com autenticação JWT stateless.
- Persistência Oracle via JDBC puro (DAO + PreparedStatement, sem ORM).
- Regras de negócio e máquinas de estado do domínio (clientes, reuniões, jobs de análise, achados, tentativas, sugestões).
- Integração HTTP com o motor de análise Python (upload, timeout longo, tratamento de falhas).
- Agregação de métricas para o dashboard (fila de achados, ranking de estratégias, tempo médio de resolução).

FORA DO ESCOPO DESTES REPOSITÓRIO

- Front-end Next.js — consome esta API, mas vive em repositório próprio.
- Motor de análise por IA (Python + Ollama) — recebe o arquivo bruto e devolve triagem/relatório.
- Infraestrutura de produção — o projeto assume o banco Oracle acadêmico da FIAP e roda localmente ou via docker-compose.

Fluxo de valor, ponta a ponta

Um usuário (vendedor/CSM) cadastra seus **clientes**, faz upload da transcrição de uma **reunião**; o motor Python analisa o conteúdo e devolve achados; esses achados viram **findings** que o usuário tenta resolver aplicando **estratégias** (registrando **tentativas**); o sistema sugere fechamentos (**suggestions**) a partir de reuniões subsequentes; e tudo isso alimenta um **dashboard** de métricas de sucesso por estratégia e por cliente.

- **Autenticação & Perfil** — cadastro e login com JWT stateless (POST /api/auth/register, /login), consulta e atualização parcial do próprio perfil (GET/PATCH /api/auth/me).
- **Gestão de Clientes** — CRUD completo de clientes por usuário, com deduplicação automática por nome normalizado (name_key) e ciclo de status prospect → ativo/inativo.
- **Upload & Análise de Reuniões** — envio da transcrição (POST /api/analysis/upload), acompanhamento do job de análise (queued → running → done/failed) e consulta do relatório final gerado pela IA.
- **Achados (Findings)** — cada problema recorrente identificado numa reunião vira um achado rastreável, com 5 estados (awaiting_strategy → in_progress → resolved | dismissed | recurred).
- **Tentativas de Resolução** — aplicação de uma estratégia a um achado, com registro de resultado (worked | partial | failed) e anotações livres.
- **Catálogo de Estratégias** — estratégias pré-definidas (ligação de acompanhamento, escalonamento técnico, revisão de pricing, treinamento) filtráveis por tipo de achado.
- **Sugestões de Fechamento** — quando uma nova reunião indica melhora ou recorrência, o sistema propõe uma decisão (confirm_resolved, still_open, recurred) para o usuário validar.
- **Dashboard de Métricas** — fila de achados abertos, achados severos sem tentativa ativa, ranking de estratégias por taxa de sucesso, tempo médio de resolução e contadores gerais — por usuário ou por cliente específico.

Principais endpoints REST

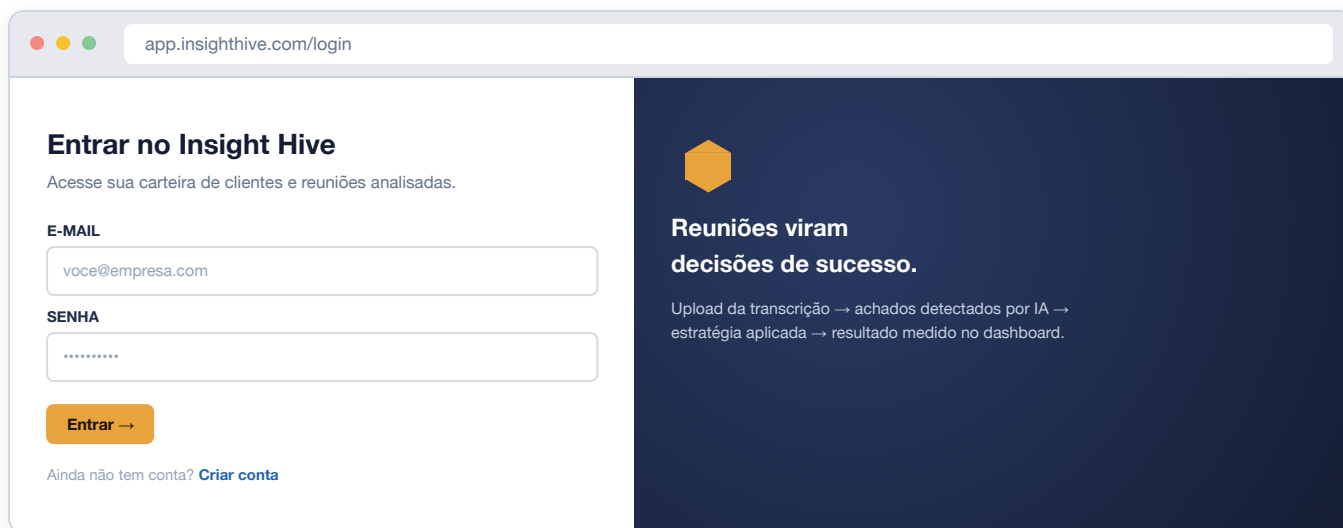
Método	Rota	Descrição
POST	/api/auth/login · /register	Autenticação pública; demais rotas exigem Bearer token.
GET/POST/PATCH	/api/clients	CRUD de clientes, com métricas de reuniões associadas.
POST	/api/analysis/upload	Envia transcrição (multipart) e dispara a análise pelo motor Python.
GET	/api/analysis/jobs/{id}	Consulta o status do job de análise (polling).
GET/POST	/api/meetings/{id}/review	Revisão humana dos eixos analisados pela IA.
GET/PATCH	/api/findings	Lista e atualiza achados por cliente.
POST/PATCH	/api/findings/{id}/attempts	Cria e conclui tentativas de resolução.
GET	/api/strategies	Catálogo de estratégias, filtrável por tipo de achado.
POST	/api/suggestions/{id}	Aplica a decisão do usuário sobre uma sugestão de fechamento.
GET	/api/dashboard	Métricas agregadas gerais ou por cliente.

Todo o JSON de entrada e saída da API usa **snake_case** (configurado globalmente via Jackson), para manter contrato direto com o front-end Next.js e com o motor Python.

3 Protótipo — Telas do Sistema

Este repositório contém apenas o backend da solução — o front-end (Next.js) que consome esta API é mantido em projeto separado. As telas a seguir são **mockups ilustrativos**, construídos a partir do contrato de endpoints descrito na Seção 2, mostrando como um cliente típico (web) percorreria o fluxo completo da solução.

3.1 — Login



app.insighthive.com/login

Entrar no Insight Hive

Acesse sua carteira de clientes e reuniões analisadas.

E-MAIL

SENHA

[Entrar →](#)

Ainda não tem conta? [Criar conta](#)

Reuniões viram decisões de sucesso.

Upload da transcrição → achados detectados por IA → estratégia aplicada → resultado medido no dashboard.

Como interagir: o usuário informa e-mail e senha (POST /api/auth/login) e recebe um token JWT, usado no header Authorization de toda chamada subsequente. Quem ainda não tem conta usa "Criar conta" (POST /api/auth/register).

3.2 — Dashboard



app.insighthive.com/dashboard

Insight Hive Dashboard

Clientes Reuniões Estratégias

ACHADOS ABERTOS 18

TENTATIVAS EM CURSO 7

TAXA MÉDIA DE SUCESSO 64%

TEMPO MÉDIO DE RESOLUÇÃO 3,2 d

Fila de achados abertos 18

- alto Cliente Fictício A — risco de churn
- médio Cliente Fictício B — lacuna de adoção
- médio Cliente Fictício C — integração pendente
- baixo Cliente Fictício D — dúvida de uso

Ranking de estratégias

- Escalar técnico
- Ligação follow-up
- Revisão de pricing
- Treinamento

Sugestões pendentes

- nova Cliente A confirmou melhora?
- nova Cliente E ainda reclama de preço
- ok Cliente F — resolvido

Como interagir: dados vêm de GET /api/dashboard (ou /api/clients/{id}/dashboard para um cliente específico). Cards de KPI e o ranking de estratégias refletem os campos calculados por DashboardService; clicar num achado da fila leva à tela de detalhe do cliente/achado.

3.3 — Clientes

app.insighthive.com/clients/128

Insight Hive Dashboard **Clientes** Reuniões Estratégias

Carteira de clientes + Novo

- ativo Fictício A Ltda.
- prospect Fictício B S.A.
- ativo Fictício C ME
- inativo Fictício D Corp.

Fictício B S.A. Nova análise de reunião

Segmento: Varejo · Porte: Médio · Contato: Maria Souza (Gerente de TI)

Reuniões recentes

kickoff_2026-08-02.vtt	2 achados
checkin_mensal_2026-07-10.vtt	1 achado
onboarding_2026-06-01.vtt	0 achados

Como interagir: a lista consome GET `/api/clients`; ao selecionar um cliente, GET `/api/clients/{id}` traz a ficha completa com as reuniões associadas. "Nova análise de reunião" abre o fluxo de upload (tela 3.4).

3.4 — Upload, Análise e Achados

app.insighthive.com/clients/128/meetings/new

Insight Hive Dashboard **Clientes** Reuniões Estratégias

Enviar transcrição — Fictício B S.A.

↑ Arraste o arquivo (.txt / .vtt) ou clique para selecionar

✓ enviado — 2 analisando — 3 concluído

Enviar para análise

Achados detectados nesta reunião

- alto Risco de churn — insatisfação com suporte
- médio Lacuna de adoção — módulo fiscal

Aplicar estratégia ao achado "Risco de churn"

Estratégia: Escalar para time técnico

Notas da tentativa...

Registrar tentativa

Como interagir: o upload chama POST `/api/analysis/upload` (multipart); o status do job é acompanhado via GET `/api/analysis/jobs/{id}` até done. Os achados listados vêm de GET `/api/findings?client_id=`; aplicar uma estratégia gera POST `/api/findings/{id}/attempts`, que move o achado para `in_progress`.

4 Modelo do Banco de Dados

A persistência é feita em **Oracle**, acessada via JDBC puro (sem JPA/Hibernate) — cada entidade de domínio tem um DAO próprio com CRUD via PreparedStatement. O DDL completo vive em `src/main/resources/schema.sql` e não é aplicado automaticamente pela aplicação: as tabelas precisam existir previamente no banco. Todas as chaves primárias usam `NUMBER GENERATED ALWAYS AS IDENTITY`.

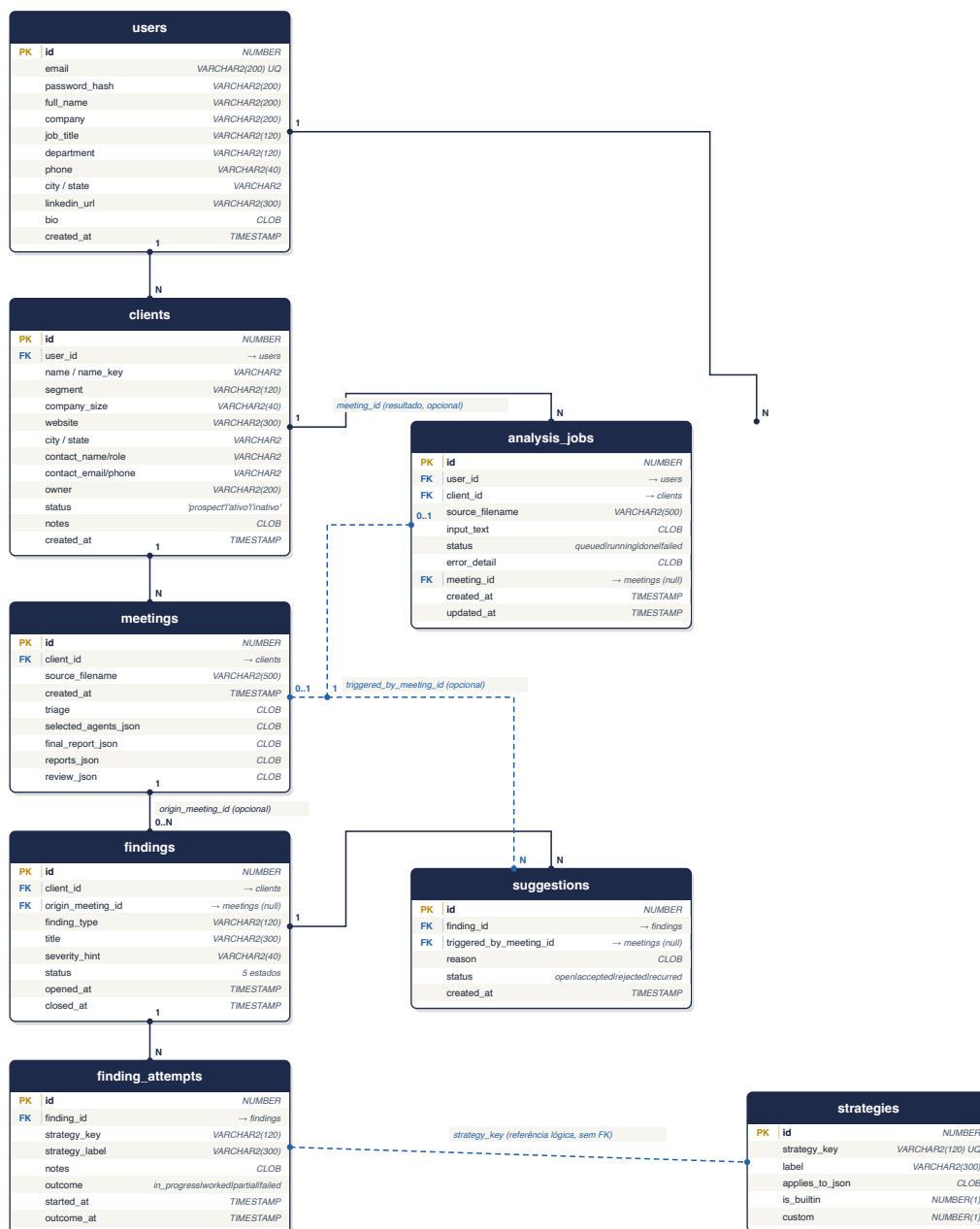


Diagrama ilustrativo do modelo relacional — chaves primárias (PK) em dourado, estrangeiras (FK) em azul; linhas tracejadas indicam relações opcionais (FK anulável) ou apenas lógicas.

Tabelas e principais constraints

Tabela	Colunas principais	Constraints / Índices
users	email (único), password_hash, full_name, dados de perfil, created_at	—
clients	user_id (FK→users, CASCADE), name, name_key, status (default prospect), dados de contato	UNIQUE(user_id, name_key); índice em user_id
meetings	client_id (FK→clients, CASCADE), source_filename, triage, selected_agents_json, final_report_json, reports_json, review_json	índice em client_id
analysis_jobs	user_id (FK, CASCADE), client_id (FK, CASCADE), status (default queued), error_detail, meeting_id (FK→meetings, SET NULL)	índices em user_id, client_id
findings	client_id (FK, CASCADE), origin_meeting_id (FK→meetings, SET NULL), finding_type, title, severity_hint, status (default awaiting_strategy)	índice em client_id
finding_attempts	finding_id (FK→findings, CASCADE), strategy_key, strategy_label, notes, outcome (default in_progress)	índice em finding_id
strategies	strategy_key (único), label, applies_to_json, is_builtin, custom	catálogo global, sem FK
suggestions	finding_id (FK→findings, CASCADE), triggered_by_meeting_id (FK→meetings, SET NULL), reason, status (default open)	índice em finding_id

Observação de modelagem: `strategies` é um catálogo compartilhado sem chave estrangeira — `finding_attempts.strategy_key` referencia uma estratégia apenas de forma lógica (validada em código, não pelo banco), permitindo que o catálogo evolua sem travar tentativas já registradas.

5 Diagrama de Classes Atualizado

As oito classes abaixo compõem o pacote `model` — POJOs simples, sem anotações JPA, montados manualmente pelos DAOs a partir do `ResultSet`. A maior parte das regras de negócio (validações e máquinas de estado) vive nos próprios métodos do modelo, não nos controllers ou services — por isso o diagrama já expõe os principais métodos de comportamento de cada classe, não apenas os atributos.

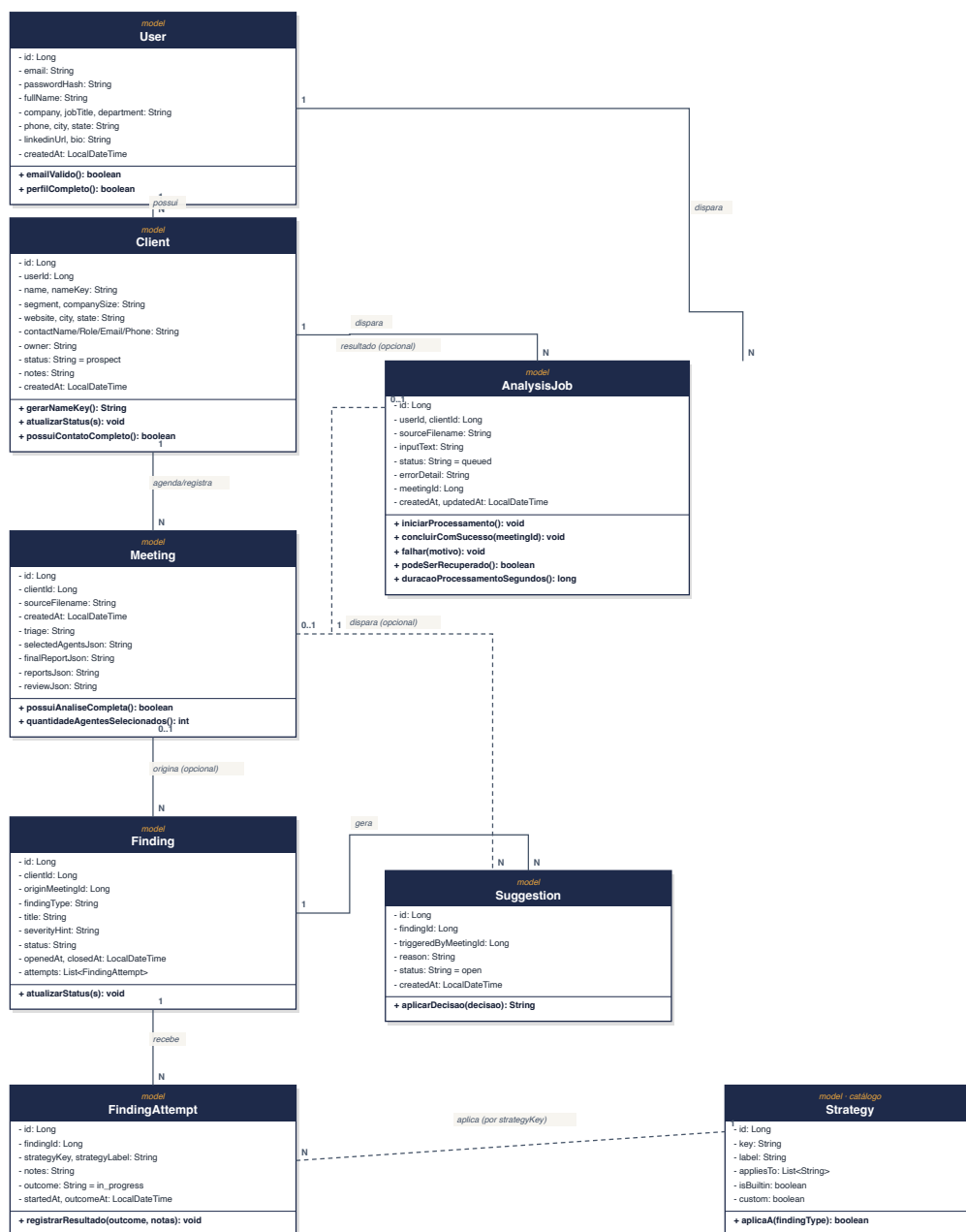


Diagrama de classes do domínio (pacote `com.sprintjava.model`) — linhas tracejadas indicam relações opcionais; a ligação `Strategy` é apenas lógica (via `strategyKey`), sem chave estrangeira.

Destaques de comportamento por classe

- **Client.gerarNameKey()** — normaliza o nome do cliente (minúsculas, sem acento, espaços colapsados) para deduplicação; garante a constraint única `UNIQUE(user_id, name_key)`.
- **AnalysisJob** — a classe com a máquina de estados mais explícita e mais coberta por testes: `iniciarProcessamento()`, `concluirComSucesso()` e `falhar()` controlam a transição `queued` → `running` → `done/failed`.
- **Finding.atualizarStatus()** — valida o novo status contra o vocabulário permitido e ajusta automaticamente `closedAt` ao resolver, dispensar ou reabrir um achado.
- **FindingAttempt.registrarResultado()** — fecha a tentativa (`outcomeAt = now()`) sempre que o resultado deixa de ser `in_progress`.
- **Strategy.aplicaA(findingType)** — decide se uma estratégia do catálogo é elegível para um tipo de achado, suportando o curinga `"*"`.
- **Suggestion.aplicarDecisao()** — traduz a decisão do usuário (`confirm_resolved`, `still_open`, `recurred`) para o novo status da sugestão, sem persistir sozinha (quem grava é o `SuggestionDAO`).

Camadas ao redor do domínio

O diagrama de classes cobre a camada de domínio; ao seu redor, o projeto segue uma separação simples em quatro camadas por pacote:

Pacote	Responsabilidade
<code>model</code>	entidades de domínio + regras de negócio e máquinas de estado (Seção 5).
<code>dao</code>	CRUD via JDBC puro (<code>PreparedStatement</code>) para cada entidade; sem pool de conexões.
<code>service</code>	agregação (<code>DashboardService</code>) e integração HTTP com o motor Python (<code>ServicoAnalisePython</code>).
<code>controller</code>	endpoints REST + DTOs de request/response (pacote <code>controller/dto</code>).
<code>security / config</code>	autenticação JWT stateless e configuração do Spring Security.